A OPERATION CORRECTNESS

When no structure modification occurs during the execution of an index access operation, i.e. insert, remove, etc., the operation is performed as if it accesses the index exclusively. In such scenario, an index traversal always descends to a correct leaf node. Both insert and remove operations (called writer in the following) acquire a write lock to prevent concurrent access by other threads to the leaf node. The only operation that poses a potential risk is lookup (denoted by reader), since the writer lacks knowledge of whether any readers are accessing the leaf node while acquiring the lock.

To guarantee reader won't retrieve a partially modified key-value pair, we employ atomic operations to perform insert and remove operations. In detail, the pointer to the key-value being inserted is stored into the *kvs* array using an atomic store operation. Similarly, when performing a remove, nullptr is atomically stored into the corresponding slot of *kvs*. Lookup operation atomically loads the pointer, thereby preventing readers from accessing a partially modified key-value.

As demonstrated above, we can ensure correctness when all operations refrain from making any structure modifications. In the following, we mainly focus on correctness when structure modification occurs. The correctness of FBTree for concurrent access with structure modification is presented in Subsection A.1, along with the demonstration of the correctness of our optimization method, cross-node tracking. Building upon this, we further discuss the correctness of latch-free update in the subsection A.2. The correctness of scan is discussed in subsection A.3 and the coordination among concurrent operations is summarized in subsection A.4.

A.1 Cross-node Tracking

The focus of our discussion will be on split, as the process involved in handling a merge bears resemblance to that of a split. Without loss of generality, we concentrate on the split operations that occur on leaf nodes. Split operations propagating to the higher-level nodes can be performed iteratively. A split operation consists of two steps: (1) move the greater half key-value pairs into the newly created node, denoted as n'; (2) insert the pointer to node n' into its parent node. A leaf node (denoted as n) is said to be undergoing a split until the pointer to the newly created node n' is inserted into its parent node.

After step (2), an index traversal could descend to node n' correctly. An incorrect leaf node only occurs when an index traversal descends to a leaf node that is undergoing a split. Given that the key-values in node n' is greater than that of the node n, it can be addressed through an alternative bypass to node n' by linking node n to n'. Hence, upon descending to a leaf node, a comparison is performed with the node's upper bound to determine whether it is necessary to proceed to the right sibling node, i.e., the newly created node n'. This is very similar to the method given by Yao et al. [28], in which the correctness has been proven for concurrent B+Tree structures with link technique.

The occurrence of structure modification is rare, however, each index traversal necessitates a costly comparison to verify any changes in the index structure. This ensures that a correct leaf node is retrieved. We aim to eliminate this unnecessary comparison. In essence, the comparison is performed to determine whether the leaf node is splitting at the moment when the pointer to the node is retrieved. Keeping track of the node and its parent can serve the same purpose, because a split involves at least the node itself and its parent node. We thus use a *splitting* bit to indicate whether the node is splitting, which is set when step (1) begins. After step (2), the *splitting* bit is unset and the *version* of the parent node is incremented. Consequently, the comparison is performed only when the *splitting* bit is set or the *version* of the parent node has changed. When a split occurs, an index traversal can detect such split by the *splitting* bit and the *version* of the parent so that the traversal can proceed to the sibling node.

For scan operations that advance through the list of leaf nodes, a similar approach, which keeps track of the current leaf node and the node to be accessed, can be used to detect whether a split occurs during a scan across nodes.

A.2 Latch-free Update

Typically, an update is performed almost identically to an insert in many index structures. Before the modification, it acquires a write lock to prevent multiple simultaneous modifications on the same key. As multiple updates on a single key-value can be serialized through CAS primitive, it can be easily implemented with latch-free updates, assuming that no concurrent structure modifications are present. The challenging problem arises when another thread concurrently performs a split operation on the leaf node addressed by an update. As a result, the key-value to be updated might have been moved to another node. Therefore, a notification mechanism is indispensable to notify the updater of potential split or merge operations performed by other threads.

Next, we address how to handle updates while considering the occurrence of node splits. We employ the node's *version* combined with atomic operation on *kvs* to fulfill that function. To implement latch-free update, an atomic exchange instruction is used to replace the pointer with nullptr and obtain the latest pointer to key-value, when moving key-value pairs to another node. The ongoing update operation thus fails because it cannot find the key-value to be updated. By checking the node's *version*, the update can detect whether a split has been performed. If the *version* changes, it performs a comparison with the upper bound (i.e. *high_key*) to determine whether the key-value is beyond current node. If the update key is larger, it proceeds to the sibling node to perform the update again, as illustrated in Figure 13. Otherwise, the update operation fails because the key-value to be updated has not been inserted yet.

A.3 Optimistic Scan

In FBTree, all of the leaf nodes are at the same level and linked as a totally ordered list. A concurrent scan operation can be performed in two steps: (1) find the start point on the leaf node list; (2) sequentially scan on the list. The former is achieved using *upper_bound* and *lower_bound* function, while the later is implemented with a concurrent iterator.

lazy rearrangement. When we cope with scan, step (1) is akin to a lookup operation, with the exception of lazy rearrangement. Since maintaining key-value order is unnecessary for both lookup and insert, key-values in a leaf node are stored out-of-order for

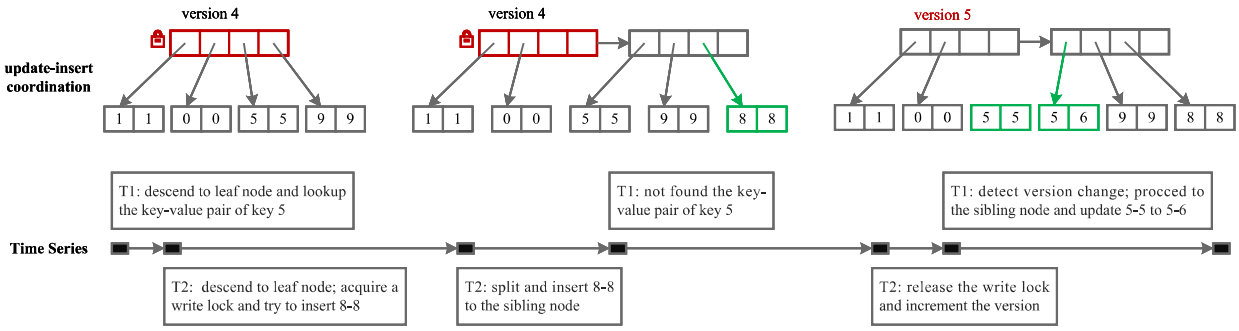

Figure 13: Update-insert coordination. Thread T1 performs an update operation and T2 performs an insert operation.

Table 2: Synchronization techniques

Operation Coordination	Optimistic Access	Write Lock	Atomic Operation
Write - Write		✓	
Update - Write			✓
Update - Update			✓
Read - Write	✓		
Read - Update			✓

improved performance. An *ordered* bit is embedded in *control* to indicate whether the key-values are in order. Upon descending to a leaf node, it checks the *ordered* bit. If the key-values are unordered, it acquires a write lock and then reorders the leaf node into an ordered arrangement. To coordinate with latch-free update, it temporarily replaces the key-value pointers with nullptr using XHG primitive.

When insert and remove operations are not considered, FBTree's concurrent iterator in step (2) is almost identical to an STL iterator. In concurrent environments, however, an iterator has to coordinate with these operations. FBTree's iterator thus contains a version to detect whether any modifications occur on the leaf node. If the version of the leaf node changes during the iterator advance, key-values will be rearranged. Next, the *upper_bound* is invoked to find the first key-value greater than the current key-value. The iterator thus can access the key-values inserted during the scan operation. Similarly, the *upper_bound* is invoked if structure modifications occur during the scan operation. The cross-node tracking technique is employed to detect structure modification when the scan operation crosses nodes. Additionally, scan operations without rearrangement coordinate with update operations via atomic operations, because an update operation does not change the arrangement of a leaf node.

A.4 Operation Coordination

In conclusion, the synchronization protocol of FBTree combines optimistic locks (write lock and optimistic access) and atomic operations (CAS, XHG, LOAD, etc.). We categorize all index operations into three distinct types: Write, Update, and Read. Write refers to the operation that modifies a node or index structure, encompassing insert, remove, split, merge, and rearrangement. Update pertains to the operation that updates the key-value pair, whereas Read denotes the operation that retrieves the key-value pair. Read includes lookup, upper_bound, lower_bound, and concurrent iterator. Table 2 illustrates how these operations are synchronized using various techniques.

Received 17 April 2024